#A Return the SHA-256 hash of the session cookie, base64url-encoded.
#B Decode the supplied token ID and compare it to the SHA-256 of the session.
#C If the CSRF token does not match the session hash, then reject the request.
#D Use the Java MessageDigest class to hash the session ID.

The `TokenController` already returns the token ID to the client in the JSON body of the response to the login endpoint. This will now return the SHA-256 hashed version, as that is what the `CookieTokenStore` returns. This has an added security benefit that the real session ID is now never exposed to JavaScript, even in that response. While you could alter the `TokenController` to set the CSRF token as a cookie directly, it is better to leave this up to the client. A JavaScript client can set the cookie after login just as easily as the API can, and as you will see in chapter 5 there are alternatives to cookies for storing these tokens. The server doesn't care where the client stores the CSRF token, so long as the client can find it again after page reloads and redirects and so on.

The final step is to update the `TokenController` token validation method to look for the CSRF token in the `X-CSRF-Token` header on every request. If the header is not present, then the request should be treated as unauthenticated. Otherwise, you can pass the CSRF token down to the `CookieTokenStore` as the `tokenId` parameter as shown in listing 4.12. If the header isn't present, then return without validating the cookie. Together with the hash check inside the `CookieTokenStore`, this ensures that requests without a